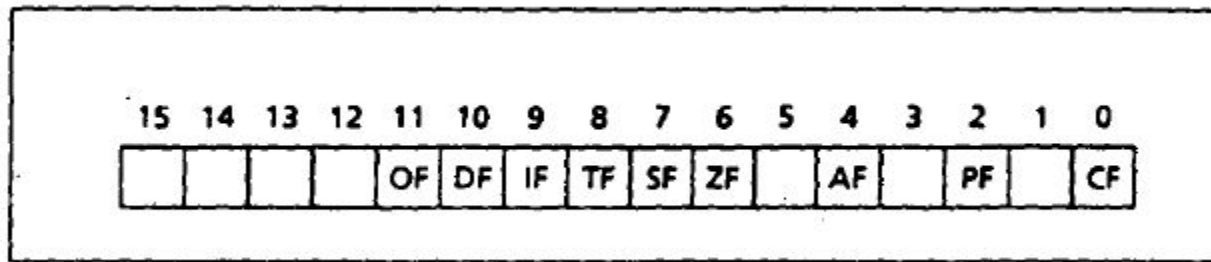


CSE 315

Microprocessors, Microcontrollers, and
Embedded Systems

Assembly Language: Arithmetic and logic instructions

The FLAGS Register



The FLAGS Register flags

Flag	Intel Mnemonic
Overflow	OF
Sign	SF
Zero	ZF
Auxiliary Carry	AF
Parity	PF
Carry	CF

Overflow Flag

- Unsigned Overflow (CF)

```
      1111 1111 1111 1111
+   0000 0000 0000 0001
-----
1 0000 0000 0000 0000
```

Overflow Flag

- Signed Overflow (OF)

$$\begin{array}{r} 0111\ 1111\ 1111\ 1111 \\ +\ 0111\ 1111\ 1111\ 1111 \\ \hline 1111\ 1111\ 1111\ 1110 = \text{FFEH} \end{array}$$

OF & CF

- $OF = 1$ for signed overflow, $CF = 1$ for unsigned overflow.
- $CF = 1$, if there is a carry out of the msb on addition.

However, CF is not affected by INC or DEC .

- $OF = 1$, if we add two numbers of the same sign and the result is of different sign. If we add two numbers of the different sign, there is no way of overflow.
 - If the carry into and out of msb don't match $OF = 1$

ZF, SF and PF

- ZF = 1 if the result is zero.
- SF = 1, if the msb of result is 1.
- PF = 1, if the *low byte* of the result has an even number of 1s (even parity).

Effect of the flags

Example 5.3 SUB AX,BX, where AX contains 8000h and BX contains 0001h.

Solution:

$$\begin{array}{r} 8000h \\ - 0001h \\ \hline 7FFFh = 0111\ 1111\ 1111\ 1111 \end{array}$$

The result stored in AX is 7FFFh.

SF = 0 because the msb is 0.

PF = 1 because there are 8 (even number) one bits in the low byte of the result.

ZF = 0 because the result is nonzero.

CF = 0 because a smaller unsigned number is being subtracted from a larger one.

Now for OF. In a signed sense, we are subtracting a positive number from a negative one, which is like adding two negatives. Because the result is positive (the wrong sign), OF = 1.

AND, OR AND XOR

- Syntax

AND destination, source

OR destination, source

XOR destination, source

- The restrictions of destination and source are the same as ADD or SUB instructions

AND, OR AND XOR

- Effect on flags
 - ✓ SF, ZF, PF reflect the result
 - ✓ AF is undefined
 - ✓ CF, OF = 0

AND, OR AND XOR

- Example

✓ XOR AX, AX  Clearing a Register

✓ OR AL, 81h  Set the MSB and LSB while preserving other bits

✓ AND AL, 7Fh  Clear the Sign Bit

81h=1000 0001

7Fh=0111 1111

AND, OR AND XOR

- Example



CMP CX,0



Testing a Register for 0



AND AL, 0Fh



Converting ASCII Digit to a number

How it Works

ASCII digits '0' through '9' are represented by the hex values 30h to 39h. By applying a bitwise AND with 0Fh (binary 0000 1111), you effectively "mask" the upper nibble and keep only the lower nibble.

Character	ASCII (Hex)	Binary	AND 0Fh	Result (Hex)
'0'	30h	0011 0000	0000 1111	00h (0)
'5'	35h	0011 0101	0000 1111	05h (5)
'9'	39h	0011 1001	0000 1111	09h (9)

Note: This method is specifically for single digits. If you need to convert a number back into ASCII for display, you would do the opposite: OR AL, 30h.

0Fh=0000 1111 NOT Instruction

- Works on a single operand
- Performs one's complement operation on the destination
- Syntax
NOT destination

TEST Instruction

- Performs similarly to the AND instruction
- Except doesn't write the output on the destination.
- Only sets or resets the flags
- Syntax
 TEST destination, source
- Usually used for flow control

TEST Instruction

- Example

✓ TEST AL,1



Testing a number is even or not

Number Type	Last Binary Bit	TEST AL, 1 Result	Zero Flag (ZF)	Next Jump Instruction
Even (e.g., 6)	Ends in 0	00000000 (Zero)	ZF = 1	JZ (Jump if Zero)
Odd (e.g., 7)	Ends in 1	00000001 (Not Zero)	ZF = 0	JNZ (Jump if Not Zero)

Shift / Rotate Instructions

- Has two possible formats:

Opcode destination, 1

Opcode destination, CL

- Effect on flags

- ✓ SF, ZF, PF reflect the result
- ✓ AF is undefined
- ✓ CF value changes according to Shift / Rotate Type
- ✓ OF =1 if result changes sign on last Shift / Rotation

⚙️ Core Mechanics

- **Left Shift:** Every bit moves to the left by one position per shift.
- **Incoming Bit:** A 0 always enters the lowest-numbered position (Least Significant Bit, or LSB).
- **Outgoing Bit:** The bit pushed out of the highest position (Most Significant Bit, or MSB) is copied into the **Carry Flag (CF)**.

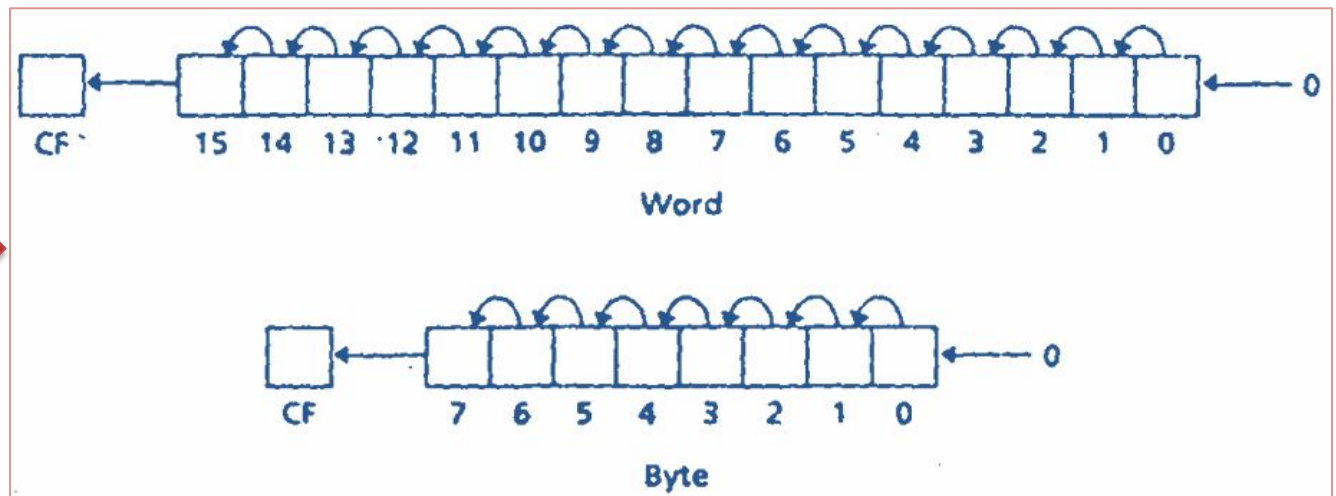
Here is a compact step-by-step breakdown of the operation `SHL DH, CL` (`DH = 8AH`, `CL = 3`):

Step	Operation Description	Binary Value of DH	Hex Value	Carry Flag (CF)
Initial	Start value before shifting	1000 1010	8AH	—
Shift 1	Shift left, insert 0 at LSB	0001 0100	14H	1
Shift 2	Shift left, insert 0 at LSB	0010 1000	28H	0
Shift 3	Final shift left, insert 0 at LSB	0101 0000	50H	0

SHL and SAL instructions

- Shifts the bits in the destination to the left
- The MSB is shifted into CF
- A 0 is shifted to LSB
- SAL and SHL generate the same machine code

SHL
and
SAL



SHL and SAL instructions

- Example

✓ If DH=8AH, CL=3

What is the result of
SHL DH, CL



DH=50H
CF=0
OF=0

✓ If DX=8AH, CX=3

What is the result of
SHL DX, CX



DX=450H
CF=0
OF=0

Register & Count	Step	Binary Representation	Hex Value	Carry Flag (CF)	Overflow Flag (OF)
DH = 8AH (8-bit) CL = 3	Initial	1000 1010	8AH	—	—
	Shift 1	0001 0100	14H	1	—
	Shift 2	0010 1000	28H	0	—
	Final	0101 0000	50H	0	0
DX = 8AH (16-bit) CX = 3	Initial	0000 0000 1000	008AH	—	—
	Shift 1	1010	0114H	0	—
	Shift 2	0000 0001 0001	0228H	0	—
	Final	0100	0450H	0	0
		0000 0010 0010			
		1000			
		0000 0100 0101			
		0000			

💡

Key Difference Notice

Notice how `DH` overflows its 8-bit boundary (losing bits to CF), whereas `DX` has 16 bits of room, so the shifted bits safely move into the upper byte (`04H`) without being lost.

The requested breakdown of the [x86 assembly multiplication slide](#) is structured below into two compact tables. [↗](#)

1. Valid Multiplication (No Sign Change)

When a bit pattern is shifted left, it acts as a **fast multiplication by 2^n** .

Example: `SHL DH, CL` where `DH = 2H`, `CL = 1` ($2 \times 2^1 = 4$)

State	Step	Binary Value of DH	Hex Value	Decimal	Carry Flag (CF)	Overflow Flag (OF)
Initial	—	0000 0010	02H	2	—	—
Final	Shift 1	0000 0100	04H	4	0	0 (Valid mathematical result)

2. Invalid Multiplication (Data Overflow)

During multi-bit shifts, the CPU flags only preserve the results of the **very last** single-bit shift operation. This behavior renders `CF` and `OF` **untrustworthy** indicators for determining if mathematical overflow occurred during execution. [↗](#)

Example: `SHL DH, CL` where `DH = 80H`, `CL = 2` ($128 \times 2^2 = 512$, which exceeds 8-bit limits)

State	Step	Binary Value of DH	Hex Value	Decimal Equivalent	Carry Flag (CF)	Overflow Flag (OF)
Initial	—	1000 0000	80H	128	—	—
Interim	Shift 1	0000 0000	00H	0 (MSB 1 dropped into CF)	1	1 (Sign changed)
Final	Shift 2	0000 0000	00H	0 (Corrupted result)	0	0 (Misleading status!)

Multiplication Using SHL and SAL instructions

- A Left shift on a binary number multiplies it by 2
- Example

✓ If DH=2H, CL=1



DH=4H
CF=0
OF=0

What is the result of
SHL DH, CL

Overflow Flag Untrustworthy During Multiplication using SHL or SAL

- Example

✓ If DH=80H, CL=2



DH=00H

CF=0

OF=0

What is the result of

SHL DH, CL

CF and OF are not reliable for multiple shifts because multiple shifts are just a series of single shifts and CF, OF only reflect the result of the last shift.

SHR vs. SAR Comparison (Single and Multiple Shifts)

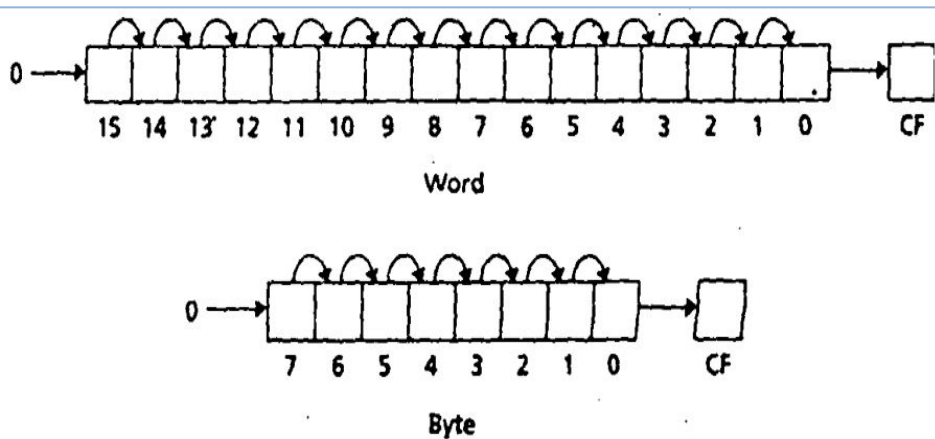
Instruction	Shift Count (CL)	Input Binary (AL = -15)	Shift Mechanism	Output Binary	Final Result (Decimal / Hex)	Flags
SHR AL, CL	1	\$1111\ 0001\$	Shifts 0 into MSB	\$0111\ 1000\$	120 (78H)	CF = 1 , OF = 1
	2	\$1111\ 0001\$	Shifts two 0s into MSB	\$0011\ 1100\$	60 (3CH)	CF = 0 , OF = undefined
SAR AL, CL	1	\$1111\ 0001\$	Preserves MSB (1)	\$1111\ 1000\$	-8 (F8H)	CF = 1 , OF = 0
	2	\$1111\ 0001\$	Preserves MSB (1)	\$1111\ 1100\$	-4 (FCH)	CF = 0 , OF = undefined

Key Mechanics

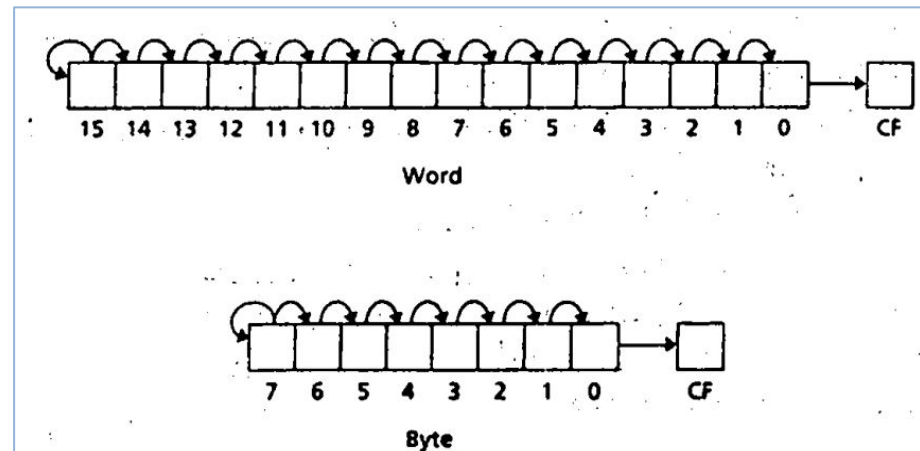
- **Logical Shift (SHR) with CL = 2 :**
 - Each shift introduces a **0** on the left.
 - For two shifts, the binary sequence changes from \$1111\ 0001\$ \rightarrow \$0111\ 1000\$ \rightarrow \$0011\ 1100\$.
 - The second bit shifted out into the Carry Flag (CF) is **0**.
- **Arithmetic Shift (SAR) with CL = 2 :**
 - The sign bit (**1**) is copied into the vacant spots to maintain the negative value.
 - For two shifts, the binary sequence changes from \$1111\ 0001\$ \rightarrow \$1111\ 1000\$ \rightarrow \$1111\ 1100\$.
 - The second bit shifted out into the Carry Flag (CF) is **0**.
- **Multi-Shift Flags:**
 - The Overflow Flag (OF) is only reliable for 1-bit shifts. It becomes **undefined** for multi-bit shifts.
 - The Carry Flag (CF) always stores the **last bit** dropped out of the right side.

SHR and SAR instructions

- Shifts the bits in the destination to the right
- The LSB is shifted into CF
- In case of SAR MSB retains its original value



SHR



SAR

SHR and SAR instructions

- SHR should be used for unsigned interpretation as it does not preserve sign
- SAR should be used for signed interpretation as it preserves sign

SHR and SAR instructions

- Example

✓ If AL=-15, CL=1

What is the result of
SHR AL, CL



AL=120H
CF=1
OF=1

✓ If AL=-15, CL=1

What is the result of
SAR AL, CL



AL=-8
CF=1
OF=1

Multiplication instructions

- **imul *source***
 - Signed multiplication
- **mul *source***
 - Unsigned multiplication
- **Byte and Word Multiplication (A X B)**
 - If two **bytes** are multiplied, the result is a 16-bit **word**
 - A: **source**
 - B: **al**
 - product: **ax**
 - If two **words** are multiplied, the result is a 32-bit ***doubleword***
 - A: **source**
 - B: **ax**
 - Product (ms 16 bits): **dx**
 - Product (ls 16 bits): **ax**

Multiplication instructions

- ***source*** can be a register or memory location (not a constant)
 - Byte form
 - $AX = AL * source$
 - Word form
 - $DX:AX = AX * source$
-
- If **ax** contains **0001h** and **bx** contains **FFFFh**
 - **mul bx;** **dx = 0000h** **ax = FFFFh**
 - **imul bx ;** **dx = FFFFh** **ax = FFFFh (-1)**

Signed Multiplication

C

Lable1:

```
short int x=0x8000;  
short int y=0xFFFF;  
x=x*y
```

Assembly

Label1:

```
MOV x, 8000H  
MOV y, FFFFH  
MOV AX, x  
MOV BX, y  
IMUL BX
```

Unsigned Multiplication

C

Label1:

```
unsigned short int x=0x8000;  
unsigned short int y=0xFFFF;  
x=x*y
```

Assembly

Label1:

```
MOV x, 8000H  
MOV y, FFFFH  
MOV AX, x  
MOV BX, y  
MUL BX
```

Multiplication instructions

- Effect on flags

- ✓ SF, ZF, AF, and PF Undefined

- CF/OF

- MUL

- 0: if upper half result 0
- 1: Otherwise

- IMUL

- 0: if upper half is sign extension of lower half.
- 1: Otherwise

More Examples

•AX=FFFFh,BX=FFFFh

Instruction	Hex Product	DX	AX	CF/OF
MUL BX	FFFE0001 (4294836225)	FFFE (!zero)	0001	1
IMUL BX	1	0000	0001	0

•AX=80h,BX=FFh

Instruction	Hex Product	AH	AL	CF/OF
MUL BX	7F80	7F(!zero)	80	1
IMUL BX	0080	00 (no sign extension)	80	1

Division instructions

▣ **cbw**

- convert byte to word

▣ **cwd**

- convert word to doubleword

▣ **div *source***

- unsigned divide

▣ **idiv *source***

- signed divide

Byte and Word Division (A/B)

- When division is performed
 - two results: the quotient and the remainder
 - Quotient and remainder are same **size** as the divisor
- For the byte form,
 - Divisor, B: **source** ;Dividend , A: **ax**
 - Quotient :**al** ;Remainder: **ah**
- For the word form,
 - Divisor, B: **source** ;Dividend , A: **dx:ax**
 - Quotient :**ax** ; Remainder: **dx**

An Example

;If **dx = 0000h**, **ax = 0005h**, and **bx = FFFEh**
(-2)

div bx; ax = 0000h dx = 0005h

idiv bx; ax = FFFEh dx = 0001h

Divide Overflow

- It is possible that the **quotient** will be too big to fit in the specified destination (**al** or **ax**)
- if the **divisor** is much smaller than the **dividend**
- the program terminates and the system displays the message "**Divide Overflow**"

Sign Extension of the Dividend

□ Word division

- The dividend is in **dx:ax** even if the actual dividend will fit in **ax**
- For **div**, **dx** should be cleared
- For **idiv**, **dx** should be made the sign extension of **ax** using **cwd**

e.g. -1250/7

```
MOV  
AX,-1250  
CWD ; sign  
extend  
MOV BX,7  
IDIV BX
```

Sign Extension of the Dividend

□ Byte division

- The dividend is in **ax** even if the actual dividend will fit in **al**
- For **div**, **ah** should be cleared
- For **idiv**, **ah** should be made the sign extension of **al** using **cbw**

References

- Ch 7, 9 Assembly Language Programming – by Yu and Marut